

# Software Relicensing: Legal Foundations, Process, Feasibility & Third-Party Impact

## Internal Research Memorandum

*Internal — For Discussion*

|              |                                    |
|--------------|------------------------------------|
| Prepared for | [Manager / Requesting Stakeholder] |
| Prepared by  | [Your Name], IT Services           |
| Date         | 29 August 2026                     |
| Version      | Draft 1.0                          |

**DISCLAIMER:** *This memorandum is general information for internal planning and does NOT constitute legal advice.*

*Copyright and licensing outcomes are fact-specific and vary by jurisdiction. Before executing any relicensing that involves third-party contributors, revenue, or copyleft-to-permissive/commercial moves, obtain a written opinion from qualified counsel.*

# 1. Executive Summary

“Relicensing” means releasing an existing body of software under different license terms going forward. Mechanically it is trivial — one commit editing a LICENSE file. Legally it ranges from a non-event to effectively impossible, and the deciding variable is almost never the technology: it is who owns the copyright in the code and whether they have already granted the rights needed to change the terms.

This memo addresses four questions the business is likely to face, in order of increasing exposure:

- 1 LEGAL FOUNDATIONS — why relicensing is a copyright question, not a Git question.
- 2 PROCESS — the concrete, repeatable steps to relicense a project we own or maintain.
- 3 FEASIBILITY & RISK — a scoring framework to judge, up front, whether a given relicensing is Easy, Hard, or Impractical, and what it would cost to attempt.
- 4 THIRD-PARTY IMPACT — what to do when a dependency WE consume changes its license out from under us (the MongoDB → Elastic → HashiCorp → Redis pattern).

■ Bottom line: For code we authored in full, relicensing is a formality. For code with outside contributors and no Contributor License Agreement (CLA), assume it is hard and budget for a consent campaign or a rewrite of hold-out contributions. For dependencies that relicense on us, our exposure is defined by HOW we use them (internal use vs. redistribution vs. hosted service).

## 2. Part A — Legal Foundations of Relicensing

### 2.1 A license is a grant of permission, not a transfer of ownership

By default, the author of code holds copyright in it automatically on creation. An open-source license does not sell or transfer that copyright; it grants everyone else a set of permissions (to use, copy, modify, distribute) subject to conditions. The copyright holder retains ownership and may additionally release the same code under other terms at any time — that is what relicensing is.

### 2.2 The core constraint: you can only relicense what you have the right to relicense

A project is rarely the work of one person. Every external contributor holds copyright in their own contribution. Unless they transferred or broadly licensed those rights, their code sits in the project under the INBOUND license only, and no one else can unilaterally change its terms. To relicense the whole codebase you therefore need, for every distinct copyright holder whose code is still present, one of: (a) their contribution was made under a permissive license that already allows sublicensing/relicensing; (b) they assigned copyright or signed a CLA granting relicensing rights; (c) they give explicit consent now; or (d) their code is removed and replaced.

### 2.3 “Inbound = Outbound” and why CLAs exist

Many projects operate on the GitHub-default “inbound=outbound” principle (codified in the GitHub Terms of Service): contributions are licensed to the project under the same license the project already uses. That is enough to keep the project running, but it grants NO extra right to change the license later. A Contributor License Agreement (CLA) or copyright assignment is the instrument that grants that extra right up front — which is precisely why every company that has successfully relicensed a popular project (MongoDB, Elastic, HashiCorp, Redis) had collected a CLA for years beforehand.

| Instrument                            | What it grants                                                               | Enables unilateral relicensing? |
|---------------------------------------|------------------------------------------------------------------------------|---------------------------------|
| Inbound=Outbound (default)            | License to the project under the existing license                            | No                              |
| DCO (Developer Certificate of Origin) | A sign-off certifying the contributor had the right to submit                | No                              |
| CLA (license grant)                   | Broad, often irrevocable license including the right to relicense/sublicense | Yes (typically)                 |
| Copyright assignment (CAA)            | Transfers ownership to the project steward                                   | Yes                             |

### 2.4 Prior releases are irrevocable

Relicensing only ever operates going forward. Every version already published under the old license stays available under that license forever — the grant to existing recipients cannot be clawed back. Anyone who obtained the code under MIT keeps their MIT rights to that version permanently, and may fork from the last old-licensed commit. This is exactly how Valkey (BSD) forked from the last BSD release of Redis, and OpenTofu from the last MPL release of Terraform.

### 2.5 Direction of travel determines difficulty

The single biggest predictor of legal difficulty is the direction of the change, because it decides whether the existing license already gives you the rights you need:

| Move                                                 | Difficulty       | Why                                                                                               |
|------------------------------------------------------|------------------|---------------------------------------------------------------------------------------------------|
| Permissive → Permissive (MIT ↔ BSD ↔ Apache-2.0)     | Low              | Permissive licenses allow sublicensing; you can re-release without polling contributors.          |
| Permissive → Source-available/Commercial (BSL, SSPL) | Low–Medium       | Sublicensing right exists; main work is drafting terms and community/PR management.               |
| Copyleft → Permissive or Commercial (GPL → MIT/BSL)  | High–Impossible  | GPL grants NO sublicensing right; you need consent from every contributor. This is the hard case. |
| Add a license option (dual-licensing)                | Low–Medium       | Often done by the owner under CLA rights; existing users unaffected.                              |
| Permissive → Copyleft (MIT → GPL)                    | Low for new work | Permissive terms allow it, but old versions remain permissive and forkable.                       |

■ The Linux kernel is the canonical “impossible” case: thousands of contributors, no CLA, many unreachable or deceased. It is frozen at GPLv2 for practical purposes — a permanent illustration of what happens without up-front rights collection.

## 3. Part B — How to Relicense a Project We Own

The following is a repeatable procedure. Steps 1–3 are analysis; steps 4–7 are execution. Do not skip the ownership audit — it is what tells you whether the rest is a day or a year.

### 3.1 Step-by-step process

- 1 **AUDIT COPYRIGHT OWNERSHIP.** Enumerate every contributor whose code is still present. Use `git shortlog -sne`, tools such as `git-quick-stats`, `licensecheck`, `ScanCode Toolkit`, or `FOSSA/Fossology`. Produce a contributor list with contribution size and current reachability.
- 2 **CONFIRM WHAT RIGHTS YOU ALREADY HOLD.** Do you have a CLA or copyright assignment on file? Is the current license permissive (sublicensable)? Either can collapse the whole effort to a formality.
- 3 **CHOOSE TARGET LICENSE & CONFIRM COMPATIBILITY.** Verify the new license is compatible with the licenses of your **DEPENDENCIES** (you cannot ship a combined work under terms that violate an incoming license — see Part D and Appendix B).
- 4 **SELECT THE CONSENT MECHANISM.** Pick per the decision tree in 3.2: rely on sublicensing rights; rely on an existing CLA; run a consent campaign; and/or rewrite hold-out code.
- 5 **COLLECT AFFIRMATIVE CONSENT** (if needed). Open a tracking issue; contact each copyright holder; record an explicit written “I agree to relicense my contributions from X to Y”. Silence is **NOT** consent. Keep the paper trail (issue links, emails, signed CLAs). Use the template in Appendix A.
- 6 **HANDLE HOLD-OUTS.** For anyone who refuses or is unreachable: identify their specific contributions and either remove/rewrite them (clean-room where prudent) or exclude those files. Document the provenance of the replacement code.
- 7 **EXECUTE & COMMUNICATE.** Update the `LICENSE` file, `SPDX` headers, package metadata (package.json, pyproject.toml, etc.), `README` badges and `NOTICE` file in one clearly-messaged commit/release. Announce the change, the rationale, the effective version, and what it means for users.

### 3.2 Decision tree (who do I need to ask?)

- Did you write 100% of the code? → No permission needed. Edit `LICENSE`, done.
- Is there a CLA/assignment covering all contributors? → The CLA authorizes it; proceed, honoring any notice obligations in the CLA.
- No CLA, but current license is **PERMISSIVE** (MIT/BSD/Apache)? → You may re-release going forward without polling contributors (old versions stay permissive).
- No CLA and current license is **COPYLEFT** (GPL/MPL/AGPL)? → You need explicit consent from every copyright holder still in the tree, or you must rewrite their parts. This is the expensive path.

### 3.3 Tooling checklist

| Need                                | Tooling                                                                                                                                                   |
|-------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Enumerate contributors              | <code>git shortlog -sne</code> ; <code>git log</code> ; GitHub contributors API                                                                           |
| Scan license headers & provenance   | <code>ScanCode Toolkit</code> , <code>licensee</code> , <code>licensecheck</code> , <code>REUSE</code> tool                                               |
| Track dependency licenses           | <code>FOSSA</code> , <code>Fossology</code> , <code>Snyk</code> , <code>pip-licenses</code> , <code>license-checker</code> (npm), <code>cargo-deny</code> |
| Manage CLAs                         | <code>CLA Assistant</code> (GitHub bot), <code>EasyCLA</code> (Linux Foundation), <code>DCO</code> bot for sign-offs                                      |
| Apply <code>SPDX</code> identifiers | <code>REUSE</code> ( <code>reuse lint</code> / <code>reuse annotate</code> )                                                                              |



## 4. Part C — Feasibility & Legal-Risk Assessment Framework

Use this to give management a fast, defensible read on any proposed relicensing before committing resources. Score the six factors, sum the points, and read the tier.

### 4.1 Scoring matrix (score each factor 0–3; higher = harder)

| Factor                   | 0 (easy)            | 1                        | 2                 | 3 (hard)                                |
|--------------------------|---------------------|--------------------------|-------------------|-----------------------------------------|
| Copyright concentration  | Sole author         | Small team               | Dozens            | Hundreds+                               |
| Rights already held      | Full CLA/assignment | Partial CLA              | DCO only          | Nothing                                 |
| Current license type     | Permissive          | Weak copyleft (MPL/LGPL) | —                 | Strong copyleft (GPL/AGPL)              |
| Direction of change      | Perm→Perm           | Perm→commercial          | Copyleft→copyleft | Copyleft→perm/commercial                |
| Contributor reachability | All active          | Mostly reachable         | Many stale        | Unreachable/anonymous/deceased          |
| Money/dispute at stake   | None                | Internal only            | Revenue-linked    | Active dispute / M&A / VC due diligence |

### 4.2 Difficulty tiers (sum of the six scores, 0–18)

| Total | Tier        | Interpretation & action                                                                                                                    |
|-------|-------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| 0–3   | TRIVIAL     | Proceed in-house. One commit; no counsel needed.                                                                                           |
| 4–8   | MODERATE    | Proceed in-house with a consent campaign; a short paid legal review of the plan is prudent.                                                |
| 9–13  | HARD        | Feasible but resource-heavy. Budget weeks–months; engage counsel; plan hold-out rewrites.                                                  |
| 14–18 | IMPRACTICAL | Assume you cannot relicense cleanly. Consider dual-licensing new work only, a fork with fresh terms, or a full rewrite. Counsel mandatory. |

### 4.3 When to spend on a lawyer (and when not to)

- SKIP counsel: sole-authored code; small reachable team; permissive→permissive; internal-only tools.
- GET counsel: copyleft→permissive/commercial; any revenue or acquisition/VC diligence depends on it; contributors you cannot fully account for; moving to source-available/commercial terms (BSL/SSPL) where wording materially affects monetization; any hint of a dispute.
- Cheap first steps before paid counsel: choosealicense.com and SPDX; the FSF and OSI relicensing guidance; the Software Freedom Law Center / Software Freedom Conservancy (sometimes free guidance to OSS projects); [opensource.stackexchange](https://opensource.stackexchange.com) for sanity checks (not legal advice).

## 5. Part D — Third-Party Relicensing: When a Dependency Changes On Us

Since 2018 a clear pattern has emerged: infrastructure vendors relicense popular permissive/copyleft projects to source-available terms (BSL, SSPL, RSAL) to stop hyperscalers reselling them as managed services. Because prior releases are irrevocable, the community response has repeatedly been a permissive fork from the last open release. As a CONSUMER of such software, our exposure depends entirely on how we use it.

### 5.1 How much does it actually affect us? (usage decides)

| How we use the dependency                                             | Typical exposure under BSL/SSPL                                                                                                |
|-----------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| Internal use only (not offered to third parties)                      | Usually little/no impact — most restrictions target offering the software AS A SERVICE to others.                              |
| Bundled/redistributed inside our product                              | Review required — source-available terms may restrict redistribution and impose conditions incompatible with our license.      |
| Offered to customers as a managed/hosted service                      | Highest impact — this is exactly the activity SSPL/BSL are written to capture; likely requires a commercial license or a fork. |
| Linked as a library into distributed code (copyleft move, e.g. →AGPL) | Review required — AGPL network-copyleft can reach across a service boundary; may force source disclosure.                      |

### 5.2 Response checklist when a dependency relicenses

- 1 IDENTIFY the exact version boundary — the last release under the old (open) license and the first under the new terms.
- 2 CLASSIFY our usage against the table in 5.1 to size the exposure.
- 3 EVALUATE the maintained permissive fork if one exists (e.g. Valkey for Redis, OpenTofu for Terraform, OpenSearch for Elasticsearch). Check governance (Linux Foundation-backed forks are lower-risk) and drop-in compatibility.
- 4 DECIDE: pin to the last open version (short-term only — no security patches long-term), migrate to the fork, or negotiate a commercial license with the vendor.
- 5 DOCUMENT the decision and update our SBOM / dependency license inventory so the next audit is clean.

### 5.3 Preventive posture

- Maintain an SBOM and automated license scanning in CI (fail the build on disallowed licenses).
- Monitor license changes of critical dependencies (watch LICENSE files / release notes; there are dedicated relicensing-alert services).
- Prefer dependencies under foundation governance (ASF, Linux Foundation, CNCF) — far less likely to be relicensed unilaterally by a single vendor.



## 6. Part E — Precedents & Case Studies

These are the reference points management will recognize. Dates verified against vendor announcements, Wikipedia and OSI records (August 2026).

| Year | Project                            | From → To                               | Why / Outcome                                                                                                                                          |
|------|------------------------------------|-----------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1999 | BSD (UC Berkeley)                  | 4-clause BSD → 3-clause BSD             | Berkeley rescinded the advertising clause (22 Jul 1999) to fix scaling pain and GPL incompatibility — the origin of modern 3-clause BSD.               |
| 2018 | MongoDB                            | AGPLv3 → SSPL                           | First of the modern wave; aimed at cloud providers reselling it. OSI later declared SSPL not open source; Debian/RHEL/Fedora dropped it.               |
| 2021 | Elastic (Elasticsearch, Kibana)    | Apache-2.0 → SSPL + Elastic License 2.0 | Targeted AWS. AWS forked to OpenSearch. Elastic re-added AGPLv3 as an option in Sept 2024.                                                             |
| 2023 | HashiCorp (Terraform, Vault, etc.) | MPL-2.0 → BSL 1.1                       | Community forked Terraform as OpenTofu (Linux Foundation). Aug 2023.                                                                                   |
| 2024 | Redis                              | BSD-3-Clause → dual SSPL/RSALv2         | 20 Mar 2024. Within days the community forked Redis 7.2 as Valkey under the Linux Foundation, keeping BSD.                                             |
| 2025 | Redis (reversal)                   | → added AGPLv3 option                   | Redis re-added an OSI-approved AGPLv3 option, partly reversing course after the Valkey exodus — shows relicensing can carry lasting reputational cost. |

■ Pattern for management: (1) the trigger is almost always a hyperscaler monetizing the project without giving back; (2) because old releases are irrevocable, a permissive FORK reliably appears within days; (3) foundation-governed forks (Valkey, OpenTofu, OpenSearch) tend to win mind-share; (4) the relicensing vendor often pays a durable reputation/adoption cost, sometimes reversing course.

## 7. Recommendations

- 1 FOR PROJECTS WE ORIGINATE: adopt a CLA (via CLA Assistant/EasyCLA) or DCO+broad-grant policy from day one. This is the single cheapest insurance against future relicensing pain.
- 2 FOR ANY PROPOSED RELICENSING: run the Part C scoring matrix first and attach it to the proposal. Do not start a consent campaign before the ownership audit (Part B, step 1).
- 3 FOR DEPENDENCIES WE CONSUME: stand up automated license scanning in CI and an SBOM; classify critical dependencies by our usage type (Part D) so we can react fast if one relicenses.
- 4 ESCALATE TO COUNSEL only at Part C tier HARD/IMPRACTICAL, or whenever revenue, redistribution, or a copyleft→permissive/commercial move is involved. Below that, in-house handling is appropriate.
- 5 PREFER foundation-governed dependencies where a choice exists, to minimize unilateral-relicensing risk.

## Appendix A — Contributor Relicensing-Consent Template

Post as a pinned tracking issue and/or email each copyright holder. Collect an explicit reply from every contributor; a thumbs-up reaction is weaker evidence than a written line of text. Keep links.

### A.1 Tracking-issue body

Title: [Relicensing] Consent needed: <OLD-LICENSE> → <NEW-LICENSE>

Body: “We intend to relicense <PROJECT> from <OLD-LICENSE> to <NEW-LICENSE>, effective from the next release. Your past contributions are covered by copyright you hold, so we need your explicit consent. If you agree, please reply on this issue with exactly: ‘I, <name / @handle>, agree to relicense my contributions to <PROJECT> from <OLD-LICENSE> to <NEW-LICENSE>.’ The checklist below tracks responses. Contributions from anyone who cannot consent will be removed or rewritten before the change.”

### A.2 Per-contributor checklist line

- [ ] @handle — <files/areas touched> — status: awaiting / AGREED (link) / declined / unreachable

### A.3 Direct email variant

“Hi <name>, you contributed to <PROJECT> (<link to commits>). We are moving it from <OLD> to <NEW> going forward and need your consent to relicense your contributions. Please reply ‘I agree to relicense my contributions from <OLD> to <NEW>.’ If we don’t hear back, we will remove or rewrite the affected code. Thank you.”

## Appendix B — License Compatibility Quick Reference

Rough one-way compatibility for combining code into a larger work. “Compatible” means the combined work can be distributed under the target column’s license. Always verify specifics; this is a heuristic, not legal advice.

| Incoming code license                | Can go into MIT/BSD work?               | Into Apache-2.0?                       | Into GPLv3 work?            |
|--------------------------------------|-----------------------------------------|----------------------------------------|-----------------------------|
| MIT / BSD-2/3-Clause                 | Yes                                     | Yes                                    | Yes                         |
| Apache-2.0                           | Yes (keep NOTICE)                       | Yes                                    | Yes (GPLv3 only, not v2)    |
| MPL-2.0 (file-level copyleft)        | Keep MPL files as MPL                   | Same                                   | Yes (MPL is GPL-compatible) |
| LGPL                                 | Dynamic-link ok; comply w/ LGPL         | Same                                   | Yes                         |
| GPLv2-only                           | Result becomes GPLv2                    | Incompatible w/ Apache-2.0             | Only if ‘or later’          |
| GPLv3 / AGPLv3                       | Result becomes GPLv3/AGPL               | One-way (Apache→GPLv3 ok, not reverse) | Yes                         |
| SSPL / BSL / RSAL (source-available) | No — not OSI-open; treat as proprietary | No                                     | No                          |

■ The 4-clause BSD advertising clause is GPL-INCOMPATIBLE — if you find it in a dependency, its presence blocks combination with GPL code. Modern 3-clause BSD does not have this problem.

## Appendix C — Glossary

| Term                          | Meaning                                                                                                             |
|-------------------------------|---------------------------------------------------------------------------------------------------------------------|
| Permissive license            | Minimal conditions; allows proprietary/relicensed derivatives (MIT, BSD, Apache-2.0).                               |
| Copyleft                      | Requires derivatives to carry the same license (GPL strong; LGPL/MPL weaker/file-scoped).                           |
| AGPL                          | GPL extended to cover use over a network — hosting counts as distribution.                                          |
| Source-available              | Source is visible but the license is not OSI-open (SSPL, BSL, RSAL). Treat like proprietary.                        |
| BSL (Business Source License) | Source-available with a Change Date after which each release converts to an open license.                           |
| SSPL                          | MongoDB's license requiring service-providers to open-source their whole service stack; not OSI-approved.           |
| CLA                           | Contributor License Agreement — grants the project broad rights, typically including relicensing.                   |
| DCO                           | Developer Certificate of Origin — a sign-off certifying the right to contribute; does NOT grant relicensing rights. |
| Inbound=Outbound              | Contributions are licensed under the project's existing license by default.                                         |
| SPDX                          | Standard identifiers/format for declaring licenses in source and SBOMs.                                             |
| SBOM                          | Software Bill of Materials — inventory of components and their licenses.                                            |
| Relicensing                   | Releasing existing code under new terms going forward; prior releases remain under old terms.                       |